

PROJECT ID:



University of Manouba
National School of Computer Science



SUMMER INTERNSHIP REPORT

Advanced AI Orchestration

Automated Conversational Evaluation Pipeline

AUTHOR

Omar Brika



COMPANY

Focus Corporation

RESPONSIBLE'S NAME

Oussama NASR

PROFESSIONAL SUPERVISOR

Ayman Mkaouar

ADDRESS

Chotrana, Ariana, Tunisia

Academic Year 2025 / 2026

Appréciations et signature de l'encadrant

DATE

SIGNATURE AND STAMP

Résumé

Valider les réponses d'un assistant vocal embarqué à grande échelle, dans de nombreuses langues et sous des contraintes de latence proches de celles d'un environnement automobile, est un problème coûteux à résoudre par des méthodes manuelles ou par des scripts de correspondance de mots-clés. Ce stage, réalisé au sein du département R&D de Focus Corporation, propose un banc de test conversationnel automatisé dans lequel un unique modèle de langage léger, exécuté localement via llama.cpp, joue tour à tour les rôles de Prompteur, de Routeur et de Juge face à un Dispositif Sous Test (DUT), le tout orchestré par un graphe LangGraph. Neuf modèles candidats (familles Qwen3.5, gemma-4, Phi et Ministral) ont été évalués sur un jeu de 216 cas de test multilingues couvrant 14 domaines automobiles, avec une instrumentation fine de la latence par type d'appel. Les résultats désignent gemma-4-E2B-it comme le meilleur compromis pour la production (95,8 % de précision, 0,734 s), Qwen3.5-4B restant préférable lorsque la précision maximale prime.

Mots-clés : orchestration de modèles de langage, LangGraph, agents conversationnels, benchmark multilingue

Abstract

Validating an in-vehicle voice assistant’s responses at scale, across many languages, and under automotive-grade latency constraints is expensive to solve with manual QA or keyword-matching scripts. This internship, carried out in Focus Corporation’s R&D department, builds an automated conversational evaluation pipeline in which a single lightweight LLM, served locally through llama.cpp, successively plays the roles of Prompter, Router, and Judge against a Device Under Test (DUT), the whole orchestrated as a LangGraph state graph. Nine candidate models (the Qwen3.5, gemma-4, Phi, and Ministral families) were benchmarked on 216 multilingual test cases spanning 14 automotive domains, with latency instrumented per call type. Results identify gemma-4-E2B-it as the best production trade-off (95.8% accuracy, 0.734 s average latency), with Qwen3.5-4B preferred when maximum accuracy is required.

Keywords: LLM orchestration, LangGraph, conversational agents, multilingual benchmark

Acknowledgements

I would like to sincerely thank Mr. Ayman Mkaouar, my professional supervisor at Focus Corporation, for his continuous guidance, his fast and detailed feedback on every iteration of the evaluation pipeline. I grateful to the Focus Corporation R&D team for their welcome and technical support throughout this internship.

Contents

Résumé	ii
Abstract	iii
Acknowledgements	iv
Contents	v
List of Figures	vii
List of Tables	viii
List of Acronyms	ix
Introduction	1
1 General Context	2
1.1 Internship Framework	2
1.2 About the Company	2
1.3 Problematic	2
1.4 Study of the Existing	3
1.5 Mission	3
1.6 Proposed Solution Diagram	4
2 State of the Art: Technologies	5
2.1 LLMs and Small Language Models	5
2.2 llama.cpp and llama-server	5
2.3 LangChain and LangGraph	6

Contents	vi
2.4 Pydantic / JSON Constrained Output	6
2.5 LangChain Callback Instrumentation	7
2.6 Edge Hardware (NVIDIA Jetson-class, CUDA 12.6)	7
3 Project Implementation	8
3.1 Graph Design.	8
3.2 Test-Suite Engineering.	9
3.3 Latency Instrumentation	10
3.4 Automation Tooling	10
3.5 The Benchmark Campaign	11
Conclusion	16
Bibliography	17

List of Figures

1.1	Proposed solution: the Prompter/DUT/Judge closed loop (the four-stage flow described above).....	4
3.1	Overall model performance: accuracy against turn-match rate, per candidate model.....	12
3.2	Latency breakdown by call type (initial prompter / routing / judgment).....	13
3.3	Per-domain accuracy across candidate models.....	13
3.4	Average per-call latency by model.....	14

List of Tables

3.1	Test-suite composition (<code>Json_samples/samples.json</code>).	9
3.2	Benchmark results across the nine candidate models.	11
3.3	Model-selection scoring: the two composite-score versions.	15

List of Acronyms

ASR	Automatic Speech Recognition
BFCL	Berkeley Function-Calling Leaderboard
CUDA	Compute Unified Device Architecture
DUT	Device Under Test
FP16	16-bit Floating-Point Precision
GGUF	GPT-Generated Unified Format — llama.cpp’s model container format
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IFEval	Instruction-Following Evaluation (benchmark)
JSON	JavaScript Object Notation
LLM	Large Language Model
LPDDR5	Low-Power Double Data Rate 5 (memory)
MMMLU	Multilingual Massive Multitask Language Understanding (benchmark)
Q4_K_M	4-bit K-quantization, medium variant (llama.cpp)
QA	Quality Assurance
TOPS	Tera-Operations Per Second
TTS	Text-to-Speech
VAD	Voice Activity Detection
VRAM	Video Random-Access Memory

Introduction

Automotive voice assistants are evaluated against hundreds of natural-language commands, in dozens of languages, before they can be shipped inside a vehicle. Historically, this evaluation has relied on human testers reading scripted scenarios aloud, or on brittle keyword-matching scripts that cannot tell a correct paraphrase from a wrong action. Neither approach scales to the combination of domains, languages, and conversational turns that a modern in-cabin assistant must support, and neither can run inside the tight latency budget of the embedded hardware the assistant will eventually ship on.

This report describes the work carried out during a first-year internship in the R&D department of Focus Corporation, whose objective was to replace that manual loop with an automated conversational evaluation pipeline. A single lightweight large language model (LLM), served locally through llama.cpp and orchestrated as a LangGraph state graph, plays three cognitive roles in sequence – Prompter, Router, and Judge – against a Device Under Test (DUT), closing the test loop without a human in it. Nine small-to-medium open-weight models were benchmarked on a purpose-built, 216-case multilingual automotive test suite, and the resulting latency and accuracy data were used to recommend a model for deployment on Focus Corporation’s target Jetson-class edge board. As a secondary, security-oriented deliverable, an authorized vulnerability assessment of that same target board was carried out, following a standard reconnaissance-enumeration-exploitation methodology.

The remainder of this report is organized as follows. Chapter 1 sets the general context of the internship and the problem it addresses. Chapter 2 reviews the state of the art of the technologies the pipeline is built on. Chapter 3 details the implementation – the orchestration graph, the test suite, the latency instrumentation, the automation tooling, and the benchmark campaign and its results. The report then closes with a discussion of what was achieved and what remains as future work, followed by Appendix A, which documents the security assessment as a self-contained deliverable.

CHAPTER 1

General Context

1.1 Internship Framework

This report covers a first-year internship completed within the curriculum of the National School of Computer Science (ENSI), University of Manouba, for the 2025/2026 academic year. The internship was hosted by the Research & Development department of Focus Corporation, based in Chotrana, Ariana, Tunisia, under the professional supervision of Mr. Ayman Mkaouar. Its scope was defined jointly by the department's need for a scalable QA process for an automotive voice assistant, and the academic requirement to apply software-engineering and AI methods to a real, open-ended problem over the course of the internship.

1.2 About the Company

Focus Corporation's RD department works on embedded systems, robotics, and artificial intelligence, developing technologies that span multiple industries and application domains. Within this broader scope, one active focus area is applied AI for the automotive industry, particularly in-vehicle voice interaction evaluating and integrating conversational AI components before they are embedded in a vehicle's infotainment stack. This report focuses on the quality-assurance stage of that pipeline: verifying a voice-assistant build's behaviour across the vehicle's supported domains and languages before it is signed off the gap this internship's pipeline addresses.

1.3 Problematic

The central challenge addressed in this work is one of scale under constraint. A production automotive voice assistant must be validated against an extensive and expanding matrix of user intents (navigation, media, climate control, phone, and others), a double-digit set of supported languages, and multi-turn conversations arising when the assistant requires clarification. Each new build necessitates re-validation against

this matrix, with the validation process required to operate within development-cycle timeframes and, ultimately, to reflect the latency constraints of the embedded, GPU-shared hardware on which the assistant will be deployed. This internship therefore sought to address the following question: how can such coverage be reproduced automatically, economically, and consistently across languages, without requiring a human tester for every domain $\times$ language $\times$ build combination?

1.4 Study of the Existing

Three existing approaches were considered and found insufficient on their own. Traditional, human-led manual QA – a human tester speaks or types each command and manually judges the assistant’s reply – remains the ground-truth method in terms of judgment quality, but it does not scale: it is slow, expensive to repeat on every build, and its throughput cannot grow with the number of supported languages and domains. Scripted keyword-matching, where a test harness checks the assistant’s transcript for expected keywords or intents, scales well and runs instantly, but it is brittle: a semantically correct paraphrase that omits the expected keyword is marked as a failure, and, more importantly, keyword matching cannot distinguish a wrong action (the assistant did something plausible but incorrect) from a hallucination (the assistant fabricated a response unrelated to the request) – both simply register as “no match.” Cloud LLM-based judging, sending the assistant’s transcript to a large hosted model for a PASS/FAIL judgment, solves the semantic-matching problem, but introduces a network round-trip and a per-call cost that make it unsuitable both for a fast, everyday grading loop and, more fundamentally, for representing the latency behaviour of the final embedded, offline deployment.

None of the three approaches doubles as a driver of the conversation itself – they all assume a human or another scripted harness is already producing the next voice command. The pipeline described in this report was designed to remove that assumption as well.

1.5 Mission

The mission set for this internship was to design and build a single, self-contained evaluation pipeline in which one lightweight LLM, running entirely on local, OpenAI-compatible inference (llama.cpp), plays all three roles a human tester or a keyword script would otherwise require: generating the next natural-language voice command (Prompter), deciding whether the Device Under Test’s (DUT) reply concludes the interaction or calls for a follow-up (Router), and issuing a semantically grounded PASS/FAIL verdict with a written justification (Judge). A second objective, added once the pipeline itself worked, was to benchmark a representative pool of small open-weight models against a purpose-built multilingual automotive test suite, and to translate

the resulting accuracy/latency data into a concrete model recommendation for Focus Corporation’s target edge hardware.

1.6 Proposed Solution Diagram

Figure 1.1 shows the closed conversational loop between the Prompter and the Judge around the Device Under Test, in four stages. Initiation: the Prompter uses the test case’s configuration (intent, target language) to generate the first natural voice command sent to the DUT. Evaluation: the Prompter ingests the DUT’s response and the accumulated conversation history. Routing: based on that analysis, the Prompter’s own decision engine determines whether the conversation requires further clarification or has reached a natural conclusion. Handoff: once the conversation is judged complete, the full state – the entire transcript – is forwarded to the Judge, which issues the final PASS/FAIL verdict.

NOTE ON THE DIAGRAM

The diagram already reflects an implementation choice made during the build: Routing is not a separate box because it is not a separate graph node in the shipped code — it is the second kind of LLM call the Prompter itself makes. This design simplification, and how it differs from the three-node architecture originally planned, is discussed in full in §3.1 (Graph Design).

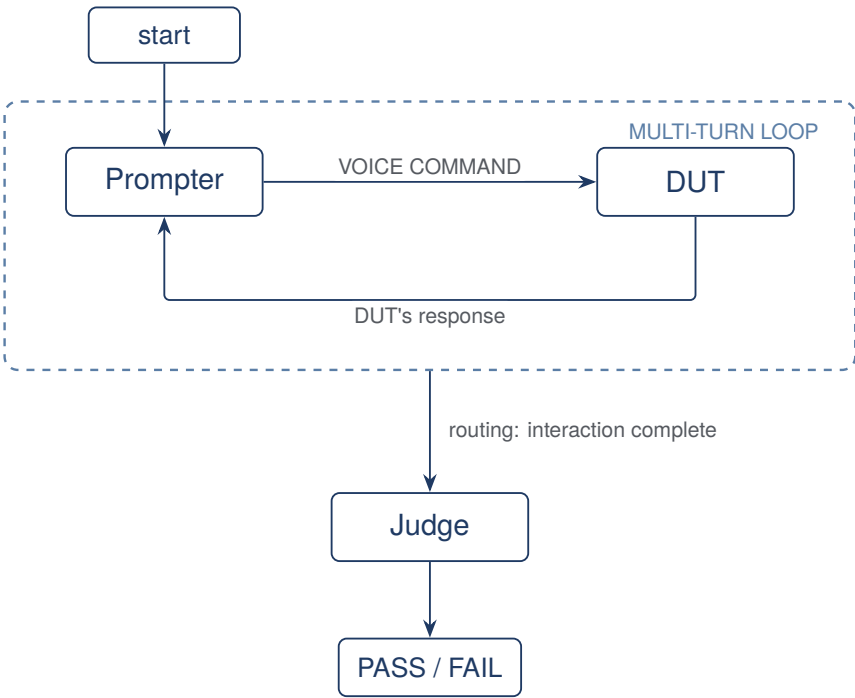


Figure 1.1. Proposed solution: the Prompter/DUT/Judge closed loop (the four-stage flow described above).

CHAPTER 2

State of the Art: Technologies

2.1 LLMs and Small Language Models

Large language models are increasingly used not only to generate free text but to act as function-calling, instruction-following controllers inside a larger system – exactly the role the Prompter, Router, and Judge need to fill here. Three families of public benchmarks were used to shortlist candidate models before the internship’s own benchmark campaign began: IFEval, which checks whether a model can respect a rigid output contract (essential, since every Prompter and Judge call must return parseable JSON); MMMLU and INCLUDE, which measure whether a model’s reasoning quality survives translation into the languages the test suite covers; and τ^2 -Bench / BFCL, which measure a model’s ability to act as an agent inside a changing conversational state – the closest public proxy for the Router’s job. Published scores gathered during candidate shortlisting (e.g. Qwen3.5-4B: IFEval 89.8, MMMLU 76.1, τ^2 -Bench 79.9; Qwen3.5-2B: IFEval 78.6, MMMLU 63.1, τ^2 -Bench 48.8) informed the initial candidate pool before empirical, task-specific numbers replaced them as the deciding factor in Chapter 3.

Running one of these models at full precision is not an option on a shared, power- and memory-constrained edge board. All nine candidate models are therefore served in Q4_K_M – a 4-bit, per-block quantization scheme (llama.cpp’s GGUF format) that keeps the most sensitive tensors at a slightly higher effective precision than the bulk of the weights. This roughly quarters the model’s memory footprint relative to FP16 at a comparatively small accuracy cost, which is what makes it realistic to keep one of these models resident, or to swap between them, inside the RAM/VRAM budget shared with the hardware’s other AI workloads.

2.2 llama.cpp and llama-server

llama.cpp is a C/C++ inference engine for GGUF-quantized models that runs efficiently on CPU and consumer/embedded GPUs alike, including the CUDA 12.6 stack quoted for the target Jetson-class board. Its `llama-server` binary exposes an OpenAI-compatible

`/v1/chat/completions` HTTP endpoint, which is the detail that makes the whole architecture possible: the pipeline’s LangChain `ChatOpenAI` client can talk to a fully local model over `http://localhost:<port>/v1` exactly as it would to a cloud provider, so nothing in `graph.py` or `nodes.py` needs to know or care that the “API” it is calling is actually a local process. `run.sh` launches `llama-server` with a 1024-token context and full GPU layer offload, waits for the server’s `/health` endpoint to report `ok` before starting the benchmark, and tears the server down again once the Python harness exits – which is also what lets the benchmark queue move from one candidate model to the next unattended.

2.3 LangChain and LangGraph

LangChain supplies the thin client layer (`ChatOpenAI`) and the callback interface the pipeline uses to talk to `llama-server` and to instrument every call. LangGraph, built on top of it, supplies the actual orchestration: a `StateGraph` is a graph of named node functions that each receive and return a partial update to a single shared, typed state object, wired together by edges that can be conditional on that state. This is the right abstraction for a conversational evaluator because the control flow is not linear – the same Prompter logic must be revisited every time the DUT asks for clarification, and the graph only exits the loop once a routing flag says the conversation is over. `graph.py` builds exactly that: two nodes (`prompter`, `judge`), wired by conditional edges that read the state’s `routing_flag` and route back into `prompter` on `AWAITING_DUT`, forward into `judge` on `PROCEED_TO_JUDGE`, or straight to `END` on `TERMINATED`. LangGraph’s `recursion_limit` (10 for the offline benchmark graph) is what protects a run from an infinite prompter-to-prompter loop if the routing logic never sets one of the terminal flags.

2.4 Pydantic / JSON Constrained Output

Both the Prompter and the Judge are asked, in their prompts, to return a single JSON object rather than free text, and `parsers.py` parses that text into `PrompterSchema`/`JudgeSchema` Pydantic models before the graph does anything with it – `JudgeSchema.verdict`, for instance, is typed as a two-value literal, so a validated instance is guaranteed to be either `PASS` or `FAIL` and nothing else. This is not the fully constrained decoding described as an aspiration in the original design notes (grammar-constrained generation that would make invalid JSON literally unreachable); the pipeline instead re-parses the model’s raw text output defensively. `extract_json_from_text()` in `parsers.py` tries three fallback strategies in turn – a fenced ````json` block, a bare `{...}` object, and a trailing-comma cleanup pass – before giving up and falling back to a naive text heuristic. That fallback chain is a pragmatic response to Q4_K_M-quantized small models, which do not always respect a JSON-only instruction as reliably as a full-precision frontier model would.

2.5 LangChain Callback Instrumentation

LangChain exposes a `BaseCallbackHandler` interface with hooks fired at defined points in an LLM call's lifecycle (`on_chat_model_start`, `on_llm_start`, `on_llm_end`, `on_llm_error`). `LLMTimeTracker` (`llm_tracker.py`) subclasses this handler to record a timestamp when a call starts and compute its exact duration when it ends, storing each call as a `duration/tokens/tags/run_id` record in a flat list – the `tags` field being the hook that lets the rest of the pipeline tell which of the three logical call types (`initial_prompter`, `routing_voice_cmd`, `judgment`) a given measurement belongs to, since every call to `llm.invoke()` in `nodes.py` is issued with a matching `tags` argument. Implementing this correctly required matching the exact keyword-only signatures LangChain's callback mixins expect – a point noted explicitly in the project's own engineering log after an initial implementation broke on a positional-argument mismatch. The tracker itself never touches durations beyond timing and tagging; the aggregation into the three named averages (`avg_initial_prompter_time`, `avg_routing_voice_cmd_time`, `avg_judgment_call_time`) is left to `reporter.py`.

2.6 Edge Hardware (NVIDIA Jetson-class, CUDA 12.6)

The pipeline's model-selection outcome is meant to run on the Jetson-class board specified by Focus Corporation's R&D team: an NVIDIA Ampere-architecture GPU (2048 CUDA cores, 64 Tensor Cores) rated at up to 275 TOPS of AI performance, a 12-core Arm CPU at 2.2 GHz, 64 GB of shared LPDDR5 RAM/VRAM, and CUDA 12.6. The specification explicitly notes that this compute is not dedicated to the voice-assistant pipeline alone but shared with the vehicle's other AI workloads – which is precisely why model size and per-call latency, not just accuracy, were tracked as first-class benchmark metrics throughout Chapter 3, rather than being an afterthought applied only at the end.

CHAPTER 3

Project Implementation

3.1 Graph Design

The evaluation state is a single typed dictionary, `BrainState` (`state.py`): `intent` and `target_language`, the running `recent_messages` transcript, the pre-scripted `dut_script` / `dut_script_idx` pair standing in for the Device Under Test, a `routing_flag` (`AWAITING_DUT` / `PROCEED_TO_JUDGE` / `TERMINATED`), the eventual `final_verdict` / `final_reasoning`, and bookkeeping fields (`turn_count`, `expected_turns`, `llm_logs`) used only for benchmark metrics.

Instead of the three node types in the original architecture brief, the shipped offline graph (`graph.py`) uses just two `StateGraph` nodes: `prompter` and `judge`. A conditional edge reads `routing_flag` after each `prompter` call – back to `prompter` on `AWAITING_DUT`, forward to `judge` on `PROCEED_TO_JUDGE`, or to `END` on `TERMINATED`; `judge` always edges to `END`. The routing decision isn’t a separate node – it is folded into the Prompter’s second LLM call (`create_prompter_node`, tagged `routing_voice_cmd`), which returns both the next command and an `is_interaction_complete` flag in one JSON object. This departs from the original “Agentic Memory & Multi-Turn Routing” milestone, which called for a standalone Router node; folding it in halves LLM round-trips per turn, at the cost of a router that cannot later be swapped for a smaller, faster model (see the Conclusion). The `system_design.svg` diagram already reflects this, showing only Prompter and Judge boxes.

Two details only surface on close reading of the code. First, `schemas.py`’s canonical `PrompterSchema` has just one field, `text_command`, but `nodes.py` still reads `parsed.is_interaction_complete` off it. The two are consistent in the working copy that produced the benchmark results below – that copy’s `schemas.py` also declares `is_interaction_complete` – but that field is missing from the copy currently committed as this report’s primary source. It is a real regression between the run and the committed version, and why every number here was re-derived from raw JSON output rather than trusted from source files. Second, the offline graph’s recursion guard is a bare `recursion_limit` of 10 with no exception handling; the live variant handles this explicitly instead.

A `live/` variant (`live_graph.py`, `live_nodes.py`, `live_state.py`) drives an actual, non-scripted DUT. It keeps the Prompter/Judge split but adds `voice_cmd` (text command $\rightarrow$ audio) and `asr` (DUT audio reply $\rightarrow$ text) nodes, both dependency-injected so the graph can point at a real TTS/ASR stack or a mock pair. `LiveConversationGraph` compiles with an in-memory checkpointer keyed by thread id, recovering the last good state on failure: `invoke()` sets `recursion_limit` to $3\times$ the max turns and catches `GraphRecursionError`, returning a **FAIL** verdict with reasoning instead of propagating the exception. The live Prompter also sidesteps the `is_interaction_complete` inconsistency entirely, inferring completion from an empty `text_command` – a cleaner contract the offline harness would benefit from adopting.

3.2 Test-Suite Engineering

The test suite (`Json_samples/samples.json`) holds 216 cases, each a `{name, intent, language, domain, dut_script, expected_verdict, expected_turns}` record. `dut_script` is a pre-written list of one or two DUT replies, letting the offline benchmark run unattended and reproducibly across nine models – the DUT’s behaviour is fixed per case, so only the Prompter/Router/Judge triangle under test can vary between runs.

Table 3.1. Test-suite composition (`Json_samples/samples.json`).

Property	Count	Share
Total test cases	216	100.0 %
Expected PASS	192	88.9 %
Expected FAIL	24	11.1 %
Multi-turn (2-turn) cases	24	11.1 %
Languages	12	—
Domains	14	—

The 216 cases split evenly across 12 languages (18 each: `ar-SA`, `de-DE`, `en-US`, `es-ES`, `fr-FR`, `it-IT`, `ja-JP`, `ko-KR`, `pt-BR`, `ru-RU`, `tr-TR`, `zh-CN`) and 14 automotive domains – `CLIMATE`, `MEDIA`, `NAVIGATION`, `PHONE` at 24 cases each, and `AMBIENT_LIGHTING`, `DIGITAL_ASSISTANT`, `DRIVING_ASSISTANCE`, `HVAC`, `MAINTENANCE`, `RADIO`, `SEAT_COMFORT`, `VEHICLE_CONTROL`, `VEHICLE_STATUS`, `WEATHER` at 12 each. All 24 two-turn cases expect **PASS**, exercising the Router’s clarification-loop logic; all 24 **FAIL** cases are single-turn.

The **FAIL** cases were engineered around a distinction raised during the internship’s engineering review: a wrong answer isn’t necessarily a hallucination, but every hallucination is a wrong answer – so the two need separate cases rather than one generic “fail” bucket. Twelve `MEDIA` cases are tagged `WrongAction`: the DUT performs a plausible but incorrect action (e.g. “Play jazz” answered with “Opening the sunroof.”) – a failure of action selection, not comprehension. Twelve `CLIMATE` cases are tagged

Hallucination: the DUT fabricates an unrelated response to a well-formed command (e.g. “Turn on the heater” answered with “I have added a space heater to your Amazon cart.”) – a failure to stay grounded at all. This distinction matters for the Judge’s reasoning field, and, downstream, for whether a model’s failures reflect a prompt-tuning problem (wrong action) or a groundedness problem (hallucination).

CLIMATE’s PASS cases were also deliberately written with implicit rather than explicit intent – “I am cold” rather than “Turn on the heater” – to test whether a model can infer the correct action from natural phrasing rather than a literal command. §3.5 shows this single design choice separates the benchmarked models most sharply.

3.3 Latency Instrumentation

Every `llm.invoke()` call in `nodes.py` is wrapped by a small tracking helper that reads the `LLMTimeTracker`’s call count before and after the call and sums the durations of whatever new entries appeared – robust to a call silently retrying internally, since every retry would still show up as its own timed entry. Each node call is issued with a tag identifying which of three metric buckets it belongs to: `initial_prompter` for the very first Prompter call in a conversation (no history yet), `routing_voice_cmd` for every subsequent Prompter call (deciding completion and generating the next command in the same call), and `judgment` for the single terminal Judge call. `runner.py` reads these per-call tags back out of each test case’s `llm_logs` to compute, per test case, `initial_prompter_time`, `routing_voice_cmd_time` and `judgment_call_time` (plus matching call counts), which `reporter.py` then sums and averages across the whole 216-case suite into the report’s headline `avg_initial_prompter_time`, `avg_routing_voice_cmd_time` and `avg_judgment_call_time` figures – the three numbers behind the latency-breakdown charts in §3.5.

3.4 Automation Tooling

`installModels.sh` reads `models.conf` – a flat `category|repo_id|filename` list, with a quant placeholder expanded to the requested quantization (Q4_K_M by default) – and pulls each GGUF file from Hugging Face, skipping files that already exist and retrying only the ones that failed, into a `model-dir/{reasoning, non_reasoning}/` split. That split matters at run time: the Qwen3.5 and gemma-4 families are hybrid reasoning models and are launched with their reasoning trace explicitly disabled for this task, while the Phi and Ministral families are launched unmodified. `run.sh` then iterates every model file in both subdirectories, starts `llama-server` for it (1024-token context, full GPU layer offload), polls its `/health` endpoint (up to 30 retries at 2 s, i.e. up to 60 s of load time – deliberately generous, since some of the larger candidate models take noticeably longer than a tighter timeout would allow to finish loading), runs the Python benchmark harness to completion, and tears the server down again before moving to

Table 3.2. Benchmark results across the nine candidate models (216 test cases, Q4_K_M, temperature 0.0). The model selected for deployment is shaded.

Model	Accuracy	Avg. latency (s)	Turn-match
Ministral-3-3B-Instruct	36.11 %	1.589	98.61 %
Ministral-3-8B-Instruct	60.65 %	2.976	100.00 %
Phi-3.5-mini-instruct	86.11 %	2.043	100.00 %
Phi-4-mini-instruct	89.81 %	0.737	100.00 %
Qwen3.5-2B	90.28 %	0.643	100.00 %
Qwen3.5-4B	96.30 %	1.156	100.00 %
Qwen3.5-9B	93.98 %	1.568	100.00 %
gemma-4-E2B-it	95.83 %	0.734	100.00 %
gemma-4-E4B-it	95.83 %	1.164	100.00 %

the next model – the loop that produced all nine `benchmark_report_*.json` files from a single, unattended invocation. A signal trap additionally ensures that both the Python harness and the `llama-server` process are killed if the whole run is interrupted, rather than leaving a server process on the port the next iteration needs.

Most of this tooling exists because of a specific, dated request from the professional supervisor rather than a design choice made in isolation. The June 29 sync asked, among other things, to add the Gemma 4 family to the testing queue, to track the three latency metrics separately (already discussed in §3.3), to grow the evaluation dataset to at least 200 samples, and to build a Bash script capable of iterating the model queue with the Python harness supporting the necessary command-line arguments. Each of those four requests maps directly onto, respectively, the `gemma-4-E2B-it` / `gemma-4-E4B-it` rows in the results table below, the `initial_prompter` / `routing_voice_cmd` / `judgment` tags described in §3.3, the 216-case `samples.json` (grown from an original 50-case pilot suite), and the `run.sh` / `model-name` / `port` / `test-samples` / `verbose` argument pairing described above.

3.5 The Benchmark Campaign

All nine models were benchmarked under identical conditions: `llama-server` on localhost, 1024-token context, Q4_K_M quantization, temperature 0.0 (deterministic decoding, so repeated runs are directly comparable), one model resident at a time, run against the full 216-case suite. Table 3.2 reports the resulting accuracy, average per-test latency, and turn-match rate for each model, and Figure 3.1 plots accuracy against turn-match visually.

Accuracy ranges from 36.1% (Ministral-3-3B) to 96.3% (Qwen3.5-4B) – a 60-point spread across nine models at broadly similar parameter counts (2B–9B), which shows how much instruction-following quality varies within the “small model” category

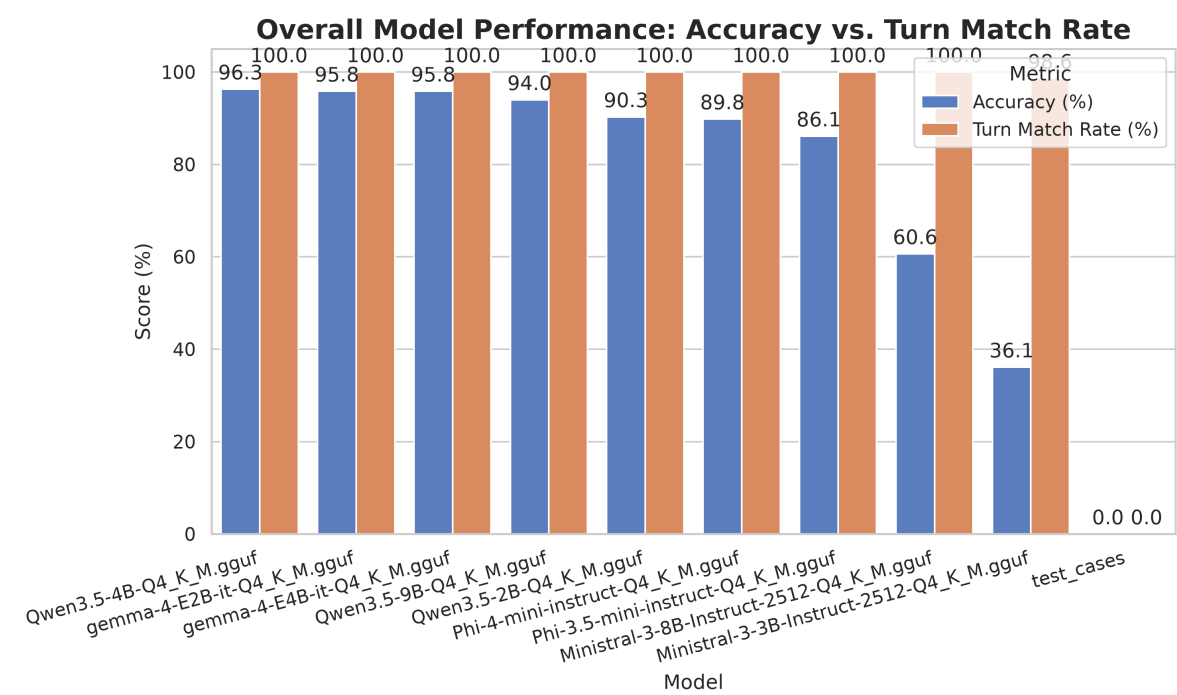


Figure 3.1. Overall model performance: accuracy against turn-match rate, per candidate model.

independently of raw size: the 9B Qwen3.5-9B (94.0%) is outperformed by its own 4B sibling (96.3%), and the 8B Ministral (60.6%) trails every Qwen3.5 and gemma-4 variant regardless of size. Turn-match – whether the model produced exactly the number of conversational turns the test case expected – is 100% for every model except Ministral-3-3B (98.6%), showing that the routing logic itself (deciding when to stop) is largely solved even for the weaker models; the accuracy gap is concentrated in the Judge’s semantic verdict and the Prompter’s action selection, not in turn-taking.

Latency is dominated by the Judge call for every model: it accounts for between 47.8% (gemma-4-E2B-it) and 63.7% (Qwen3.5-4B) of a test case’s total LLM time, computed directly from each model’s reported judgment and total call times, because the Judge is the only call given the full conversation history and asked to produce free-form reasoning before its verdict, rather than a short, templated command. Figures 3.2 and 3.4 show this breakdown per model; the two Ministral models stand out with a disproportionate routing-call cost, consistent with the accuracy problems these models show throughout the suite.

Domain-level accuracy (Figure 3.3) is where the benchmark discriminates most clearly between the viable and non-viable candidates. Safety- and comfort-adjacent domains that admit only one plausible correct action per intent – HVAC, DRIVING_ASSISTANCE, AMBIENT_LIGHTING, SEAT_COMFORT – sit at or near 100% for every model from Phi-3.5-mini upward; only the two Ministral models fall meaningfully short there (e.g. DRIVING_ASSISTANCE: 8.3% for Ministral-3-3B against 100% for six of the other seven models). CLIMATE is the clearest

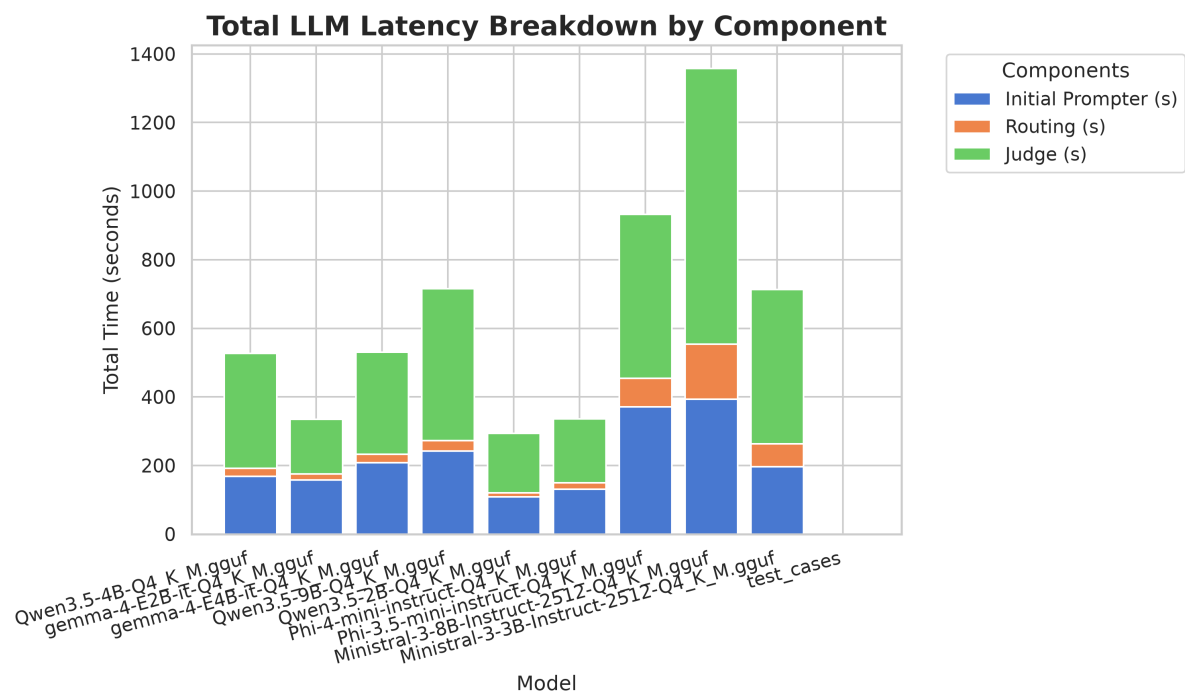


Figure 3.2. Latency breakdown by call type (initial prompter / routing / judgment).

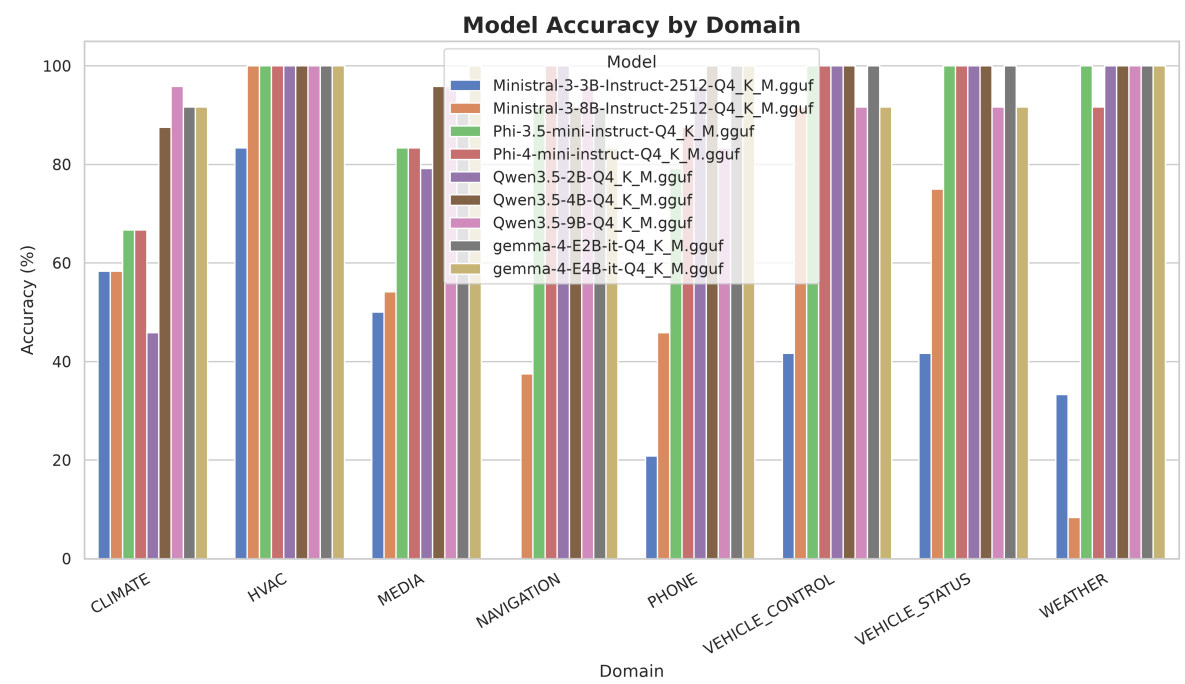


Figure 3.3. Per-domain accuracy across candidate models.

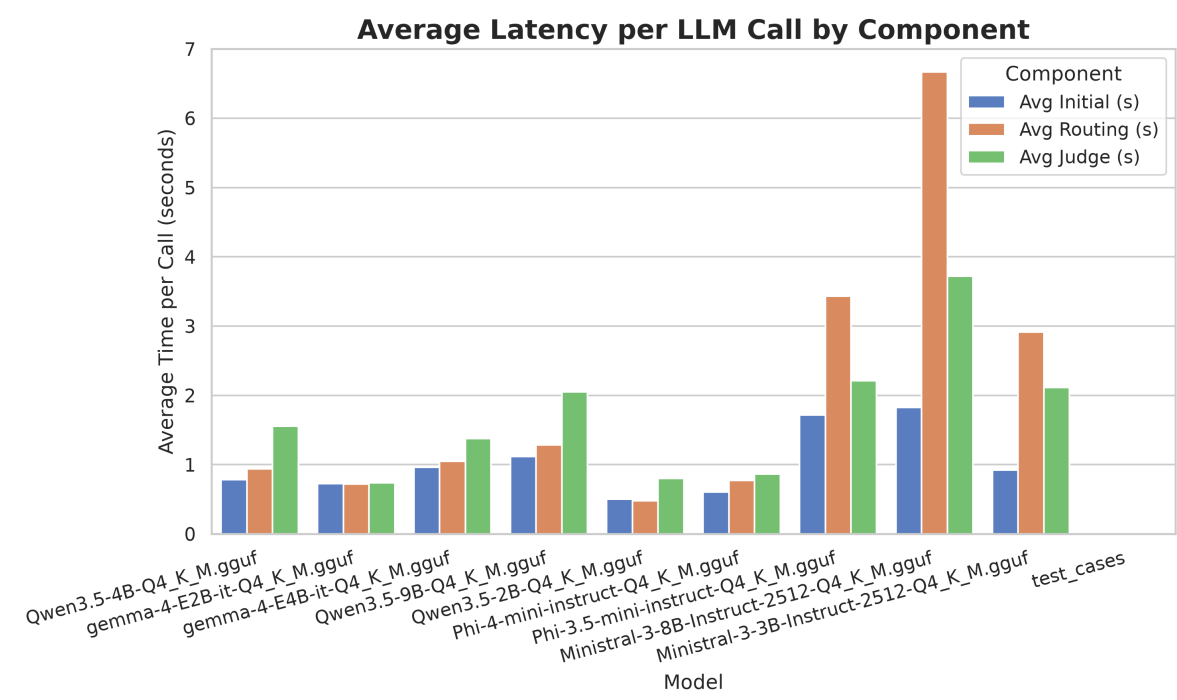


Figure 3.4. Average per-call latency by model.

differentiator among the otherwise-strong candidates, specifically because its PASS cases use implicit rather than explicit intent phrasing (§3.2): Qwen3.5-2B drops to 45.8% on CLIMATE alone despite being the second-fastest model overall, while Qwen3.5-4B (87.5%), gemma-4-E2B-it and gemma-4-E4B-it (91.7% each) and Qwen3.5-9B (95.8%) handle the same cases comfortably – meaning a deployment that leans on Qwen3.5-2B for its speed advantage would need either additional prompt engineering around implicit climate commands or a dedicated intent-classification pre-step to close that specific gap. MAINTENANCE is a second, milder weak spot across the whole board (peaking at 91.7% for the strongest models, and as low as 41.7–50% for the Phi family), suggesting the domain’s phrasing or vocabulary is generally harder for this model class regardless of family.

Two scoring passes were run over this same data to reach the final model selection, differing only in whether the Judge call’s latency is counted. Both use the same composite formula:

```
norm_speed    = 1 - (latency - min_latency) / (max_latency - min_latency)
norm_accuracy = (accuracy - min_accuracy) / (max_accuracy - min_accuracy)
jetson_score  = 0.6 × norm_speed + 0.4 × norm_accuracy
```

Version A restricts “latency” to the production-relevant calls only (initial prompter + routing, excluding the Judge, which never runs in a deployed assistant) and ranks Qwen3.5-2B first (0.980 s, score 0.960), narrowly ahead of gemma-4-E2B-it (1.450 s, score 0.959). Version B instead uses the full end-to-end latency, including the Judge

Table 3.3. Model-selection scoring: the two versions of the composite `jetson_score`, which differ only in whether the Judge call’s latency is counted.

Version	Latency scope	Rank 1	Latency	Score
A	Production only (no Judge cost)	Qwen3.5-2B	0.980 s	0.960
B	Full end-to-end (incl. Judge)	gemma-4-E2B-it	0.734 s	0.973

call, and inverts that ranking: `gemma-4-E2B-it` comes first (0.734 s total, score 0.973) because its Judge call is the fastest and most uniform of any candidate (0.729 s init / 0.722 s route / 0.740 s judge – the three components within 20 ms of each other), ahead of `Qwen3.5-2B` (0.643 s total but a comparatively heavier 0.802 s judge call, score 0.960).

The two `Ministral` models and `Phi-3.5-mini` are excluded from consideration on accuracy and latency grounds respectively (36–61% accuracy for `Ministral`; a 2.0–5.2 s latency for `Phi-3.5-mini` that both scoring versions penalize heavily), leaving six viable candidates. Among those, **`gemma-4-E2B-it` was selected as the model for deployment**: 95.8% accuracy, 0.734 s average end-to-end latency, the most uniform per-call latency profile of any candidate, and, at 2B parameters, the smallest realistic memory footprint among the models that clear 95% accuracy. It ranks first under Version B’s full end-to-end scoring (0.973) and a close second under Version A (0.959) – making it the strongest candidate regardless of whether the Judge call is counted as production latency, which was the deciding factor over `Qwen3.5-2B`’s narrow Version A lead.

NOTE ON THE MEMORY-FOOTPRINT ARGUMENT

This last point is an engineering judgment call rather than a script output: neither `jetson_model_selector.py` nor its judge-aware counterpart computes an actual VRAM budget. The ~1.5 GB (2B-class) versus ~6.5 GB (9B-class) estimates used to argue that `Qwen3.5-9B` does not comfortably fit the target board’s shared memory budget are hand-derived from parameter count and `Q4_K_M`’s per-parameter footprint, not measured by the scripts themselves.

`Qwen3.5-4B` was the closest runner-up and remains a documented fallback should the deployment later accept roughly double the latency (1.16–1.72 s depending on scoring version) in exchange for the highest raw accuracy observed in the campaign (96.3%). It was not selected because `gemma-4-E2B-it`’s latency and memory-footprint advantage was judged more valuable for the target embedded, GPU-shared hardware than the additional 0.5 accuracy points.

Conclusion

This internship addressed a scaling problem: validating an automotive voice assistant’s responses across many domains and languages without relying on a human tester or a brittle keyword script. The core deliverable is the `live/` pipeline (`live_graph.py`, `live_nodes.py`, `live_state.py`), which drives an actual, non-scripted Device Under Test through real text-to-speech and speech-recognition modules – `voice_cmd` turning the Prompter’s text command into audio, and `asr` turning the DUT’s spoken reply back into text – rather than replaying a pre-written `dut_script`. Both are dependency-injected, so the same graph can be pointed at a real TTS/ASR stack in production or at a mock pair for local testing, and the live Prompter node closes out each turn on its own signal (an empty `text_command`) rather than depending on any test-suite bookkeeping. The 216-case multilingual suite and the nine-model benchmark it produced served a narrower, supporting purpose: selecting which local LLM the live pipeline should run on. That benchmark identified gemma-4-E2B-it as the model carried into the live pipeline, with Qwen3.5-4B documented as a fallback where deployments can trade roughly double the latency for higher accuracy – but the benchmark and its offline harness were never the deliverable, only the methodology used to reach one. The engineering process behind that selection was driven throughout by concrete, dated feedback – the three-metric latency split, the 216-case dataset target, the Gemma 4 family, the PASS/FAIL hallucination-vs-wrong-action distinction – each traceable to a specific line in the pipeline.

The clearest lesson from the implementation was architectural rather than purely a modeling one: keeping `voice_cmd` and `asr` as dependency-injected callables, rather than hard-coded against a specific TTS/ASR provider, is what let the same Prompter/Judge graph that drove the offline benchmark be pointed at a real voice stack with no change to its core logic. The benchmark results validated which model to run; the dependency-injection boundary is what made that model usable in a live assistant rather than confined to the scripted evaluation harness it was selected through.

Bibliography

- [1] Qwen Team, Alibaba. *Qwen3.5 Technical Report*. 2025. URL: <https://huggingface.co/Qwen>.
- [2] Google DeepMind. *Gemma 4: Open Models Technical Report*. 2025. URL: <https://huggingface.co/google>.
- [3] Microsoft. *Phi-4-Mini Technical Report*. 2025. URL: <https://huggingface.co/microsoft>.
- [4] Mistral AI. *Ministral 3: On-Device Language Models*. 2025. URL: <https://huggingface.co/mistralai>.
- [5] Gerganov, Georgi and Contributors. *GGUF File Format Specification*. GitHub. 2023. URL: <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>.
- [6] Hugging Face. *Hugging Face Hub*. 2024. URL: <https://huggingface.co/>.
- [7] LangChain AI. *LangChain: Building Applications with LLMs*. 2024. URL: <https://python.langchain.com/>.